Eta Codegen Specification

Thomas Lu (netid:cl2625)

May 7, 2026

Overview

This specification formalizes the translation from a lowered IR of the Eta language to an abstract assembly based on the x86-64 ISA. Recall that our lowered IR has the following abstract syntax:

$$
\begin{array}{ll}
cu ::= dd_1 \ldots dd_n \; fd_1 \ldots fd_m & \text{(compilation unit)} \\
dd ::= \mathsf{data}\; L : (n_1, \ldots, n_k) & \text{(data declaration)} \\
fd ::= \mathsf{func}\; L : \sigma_1 \, \sigma_2 \ldots \sigma_n & \text{(function declaration)}
\end{array}
$$

$$
\begin{array}{ll}
\sigma ::= & \text{(lowered statement)} \\
\quad \mid t \leftarrow \epsilon & \\
\quad \mid [\epsilon_1] \leftarrow \epsilon_2 & \\
\quad \mid \mathsf{call}_m \; (\&L) \; (\epsilon_1, \ldots, \epsilon_n) & \\
\quad \mid L : & \\
\quad \mid \mathsf{jump}\; \&L & \\
\quad \mid \mathsf{cjump}\; \epsilon \; L & \\
\quad \mid \mathsf{return}\; \epsilon_1, \ldots, \epsilon_n &
\end{array}
$$

$$
\begin{array}{ll}
\epsilon ::= & \text{(lowered expression)} \\
\quad \mid n & \\
\quad \mid t & \\
\quad \mid \&L & \\
\quad \mid \epsilon_1 \; \mathsf{op} \; \epsilon_2 & \\
\quad \mid [\epsilon] &
\end{array}
$$

Now we describe the syntax of abstract assembly. An abstract assembly program is simply a sequence of abstract assembly commands, each of which takes the following form:

$$
\begin{array}{ll}
c ::= & \text{(commands)} \\
\quad \mid \mathsf{.directive}\; \mathsf{arg}_1 \ldots \mathsf{arg}_n & \text{(directives)} \\
\quad \mid L : & \text{(labels)} \\
\quad \mid \mathsf{opcode} \mid \mathsf{opcode}\; a_1 \mid \mathsf{opcode}\; a_1, a_2 & \text{(operations)}
\end{array}
$$

$$
\begin{array}{ll}
a ::= & \text{(operands)} \\
\quad \mid n & \text{(integer constants)} \\
\quad \mid r & \text{(registers)} \\
\quad \mid t & \text{(temporaries)} \\
\quad \mid \mathsf{size}\; [\mathsf{base} + \mathsf{idx} * \mathsf{scale} + \mathsf{offset}] & \text{(memory operands)}
\end{array}
$$

$$
\begin{array}{rl}
\mathsf{size} ::= & \texttt{BYTE PTR} \mid \texttt{WORD PTR} \mid \texttt{DWORD PTR} \mid \texttt{QWORD PTR} \\
\mathsf{base} ::= & n \mid r \mid t \mid L \\
\mathsf{idx} ::= & r \mid t
\end{array}
$$

$$\begin{aligned}\mathsf{scale} &::= 1 \mid 2 \mid 4 \mid 8\\ \mathsf{offset} &::= n\end{aligned}$$

We discuss the undefined metavariables that appeared in the above syntax. The metavariable .directive stands for the directives used by the assembler, such as `.text` and `.global`. The metavariable opcode stands for any valid x86-64 opcode, including `cmp`, `add`, or `push`. The metavariable t denotes an unbounded number of temporaries that share the same namespace with the temporaries in IR. They are not to be confused with r, the metavariable for the concrete registers in x86-64 such as `rax` and `r8`. We also note that, although in the above grammar, the definition of memory operands is complete and includes all four components, we might omit certain components such as index and offset in the translations below (e.g. just writing [base] or [base + scale], as is permitted by Intel syntax.

Compiling Toplevel Nodes

Same as the last spec, we use $\mathcal{T}[\![\cdot]\!]_A$ where A is the nonterminal symbol of an IRNode to denote the translation function on that specific class of IRNode to a sequence of abstract assembly statements. The translation of a compilation unit and data declaration mainly consists of various metadata used by the assembler and linker:

$$\begin{aligned}\mathcal{T}[\![dd_1 \dots dd_n\ fd_1 \dots fd_m]\!] = &\texttt{.intel_syntax noprefix}\\ &\mathcal{T}[\![dd_1]\!]_{dd}\\ &\vdots\\ &\mathcal{T}[\![dd_n]\!]_{dd}\\ &\mathcal{T}[\![fd_1]\!]_{fd}\\ &\vdots\\ &\mathcal{T}[\![fd_n]\!]_{fd}\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![\mathsf{data}\ L : (n_1, \dots, n_k)]\!] = &\texttt{.data}\\ &\texttt{.type } L\texttt{, @object}\\ &\texttt{.size } L\texttt{, } 8k\\ &L:\\ &\texttt{.quad } n_1\\ &\vdots\\ &\texttt{.quad } n_k\end{aligned}$$

For functions, in addition to metadata, we also need to allocate the stack and read the arguments into the designated temporaries for them. If a function has no more than 2 return values, we translate the function such that

$$\begin{aligned}\mathcal{T}[\![\mathsf{func}\ L : \sigma_1\ \sigma_2 \dots \sigma_n]\!]_\sigma = &\texttt{.text}\\ &\texttt{.globl } L\\ &\texttt{.type } L\texttt{, @function}\\ &L:\\ &\texttt{enter } \mathsf{frame_size_of}(L), 0\\ &\texttt{mov } _\mathsf{ARG1}, \mathsf{loc_of_arg}(1)\\ &\vdots\\ &\texttt{mov } _\mathsf{ARG}k, \mathsf{loc_of_arg}(k)\\ &\mathcal{T}[\![\sigma_1]\!]_\sigma\end{aligned}$$

$$\vdots$$
$$\mathcal{T}[\![\sigma_n]\!]_\sigma$$

where k is the number of args of the function with label L. Alternatively, if the function has more than 2 return values, we need to read in the pointer to the start of the memory region where the extra return values will be stored from the first register for arguments.

$$\begin{aligned}
\mathcal{T}[\![\mathsf{func}\ L : \sigma_1\ \sigma_2\ ...\sigma_n]\!]_\sigma = & \texttt{.text} \\
& \texttt{.globl}\ L \\
& \texttt{.type}\ L\texttt{, @function} \\
& L: \\
& \texttt{enter}\ \mathsf{frame_size_of}(L), \texttt{0} \\
& \texttt{mov}\ _\mathsf{RVLOC}, \mathsf{loc_of_arg}(1) \\
& \texttt{mov}\ _\mathsf{ARG1}, \mathsf{loc_of_arg}(2) \\
& \qquad\vdots \\
& \texttt{mov}\ _\mathsf{ARG}k, \mathsf{loc_of_arg}(k+1) \\
& \mathcal{T}[\![\sigma_1]\!]_\sigma \\
& \quad\vdots \\
& \mathcal{T}[\![\sigma_n]\!]_\sigma
\end{aligned}$$

We made use of two helper functions: The first, $\mathsf{frame_size_of}(L)$, is the number of bytes needed for the stack frame of L. Since we are currently translating to abstract assembly and not yet doing register allocation, we can leave out the implementation of this function for now. The second, $\mathsf{loc_of_arg}(n)$, is the abstract assembly operand that stores the value of the nth argument on the stack. In accordance of the Linux x86-64 calling convention, we implement this as

$$\begin{aligned}
&\mathsf{loc_of_arg}(1) = \texttt{rdi} && \mathsf{loc_of_arg}(2) = \texttt{rsi} && \mathsf{loc_of_arg}(3) = \texttt{rdx} \\
&\mathsf{loc_of_arg}(4) = \texttt{rcx} && \mathsf{loc_of_arg}(5) = \texttt{r8} && \mathsf{loc_of_arg}(6) = \texttt{r9}
\end{aligned}$$

$$\mathsf{loc_of_arg}(i) = \texttt{QWORD PTR[rbp+}8(i-7+2)\texttt{]} \qquad \text{for all } i \geq 7$$

where the +2 in the offset is because `[rbp]` and `[rbp+8]` contains the saved frame pointer and instruction pointer when the arguments are stored, respectively.

Compiling Statements

Now we translate statements. In the translations, we make use of a special translation function $\mathcal{T}[\![t, \epsilon]\!]$ which is a sequence of abstract assembly statements that has the effect of computing the IR expression ϵ and storing the result in the temporary t. Using this function, the assignment statements are straightforward but might introduce a new temporary t':

$$\mathcal{T}[\![t \leftarrow \epsilon]\!]_\sigma = \mathcal{T}[\![t, \epsilon]\!]$$

$$\begin{aligned}
\mathcal{T}[\![[\epsilon_1] \leftarrow \epsilon_2]\!]_\sigma = & \mathcal{T}[\![t_1, \epsilon_1]\!] \\
& \mathcal{T}[\![t_2, \epsilon_2]\!] \\
& \texttt{mov QWORD PTR[}t_1\texttt{]}, t_2
\end{aligned}$$

A label in the IR is just a label in abstract assembly, except we need to prepend the label with a dot. This is so that it becomes an assembler label to differentiate them with global labels, as required by the GNU Assembler on Ubuntu:

$$\mathcal{T}[\![L :]\!]_\sigma = \texttt{.}L\texttt{:}$$

We now move on to jump instructions.

$$\mathcal{T}[\![\mathsf{jump}\,\&L]\!]_\sigma = \texttt{jmp}\ .L$$

$$\begin{aligned}\mathcal{T}[\![\mathsf{cjump}\,\epsilon, \&L]\!]_\sigma = \mathcal{T}[\![t, \epsilon]\!]\\ \texttt{cmp}\,t, 0\\ \texttt{jne}\ .L\end{aligned}$$

Now we translate function calls and function return instructions, which are more complicated. We first consider function calls whose return value is no more than 2. We need to perform the following tasks in order: insert a padding of 8 bytes if needed; push the extra arguments onto the stack; save the first six arguments into the corresponding registers; transfer control to the function; save the return values into the corresponding temporaries; return the stack pointer to the state before the function call.

$$\begin{aligned}\mathcal{T}[\![\mathsf{call}_m\,(\&L)\,(\epsilon_1, ..., \epsilon_n)]\!]_\sigma = \mathcal{T}[\![t_1, \epsilon_1]\!]\\ \vdots\\ \mathcal{T}[\![t_n, \epsilon_n]\!]\\ \mathsf{insert_padding}(\max\{0, n-6\}+1)\\ \texttt{push}\ t_n\\ \vdots\\ \texttt{push}\ t_7\\ \texttt{mov}\ \mathsf{loc_of_arg}(6), t_6\\ \vdots\\ \texttt{mov}\ \mathsf{loc_of_arg}(1), t_1\\ \texttt{call}\ L\\ \texttt{mov}\ _\mathsf{RV1}, \mathsf{loc_of_rv}(1)\\ \vdots\\ \texttt{mov}\ _\mathsf{RV}m, \mathsf{loc_of_rv}(m)\\ \texttt{add}\ \texttt{rsp}, 8\max\{0, (n-6)\}\\ \mathsf{remove_padding}(\max\{0, (n-6)\}+1)\end{aligned}$$

For functions that return more than two arguments, we need to pass in the location of the region where the return values are stored before all other Eta arguments and collect the return values after the function call:

$$\begin{aligned}\mathcal{T}[\![\mathsf{call}_m\,(\&L)\,(\epsilon_1, ..., \epsilon_n)]\!]_\sigma = \mathcal{T}[\![t_1, \epsilon_1]\!]\\ \vdots\\ \mathcal{T}[\![t_n, \epsilon_n]\!]\\ \mathsf{insert_padding}(\max\{0, n-5\}+(m-2)+1)\\ \texttt{sub}\ \texttt{rsp}, 8(m-2)\\ \texttt{mov}\ \mathsf{loc_of_arg}(1), \texttt{rsp}\\ \texttt{push}\ t_n\\ \vdots\\ \texttt{push}\ t_6\\ \texttt{mov}\ \mathsf{loc_of_arg}(6), t_5\\ \vdots\end{aligned}$$

$$
\begin{aligned}
&\texttt{mov}\ \ \mathsf{loc_of_arg}(2), t_1\\
&\texttt{call}\ \ L\\
&\texttt{add}\ \ \texttt{rsp}, 8\max\{0, n-5\}\\
&\texttt{mov}\ \ _\mathsf{RV1}, \mathsf{loc_of_rv}(1)\\
&\texttt{mov}\ \ _\mathsf{RV2}, \mathsf{loc_of_rv}(2)\\
&\texttt{pop}\ \ _\mathsf{RV3}\\
&\quad\vdots\\
&\texttt{pop}\ \ _\mathsf{RV}m\\
&\mathsf{remove_padding}(\max\{0, n-5\} + (m-2) + 1)
\end{aligned}
$$

The helper function for adding and removing the padding are defined such that

$$
\mathsf{insert_padding}(k) = \begin{cases} \texttt{sub rsp}, 8 & \text{if } k \text{ is odd} \\ \texttt{nop} & \text{otherwise} \end{cases}
\qquad
\mathsf{remove_padding}(k) = \begin{cases} \texttt{add rsp}, 8 & \text{if } k \text{ is odd} \\ \texttt{nop} & \text{otherwise} \end{cases}
$$

The helper function to compute the location where the return values should be stored is defined such that

$$
\mathsf{loc_of_rv}(1) = \texttt{rax} \qquad\qquad \mathsf{loc_of_rv}(2) = \texttt{rdx}
$$

$$
\mathsf{loc_of_rv}(i) = \texttt{QWORD PTR[}_\mathsf{RVLOC} + 8(i-3)\texttt{]} \qquad \text{for all } i \geq 3
$$

We finally translate return statements.

$$
\begin{aligned}
\mathcal{T}[\![\mathsf{return}\ \epsilon_1, ..., \epsilon_n]\!]_\sigma = \ & \mathcal{T}[\![t_1, \epsilon_1]\!]\\
& \vdots\\
& \mathcal{T}[\![t_n, \epsilon_n]\!]\\
& \texttt{mov}\ \ \mathsf{loc_of_rv}(1), t_1\\
& \vdots\\
& \texttt{mov}\ \ \mathsf{loc_of_rv}(n), t_n\\
& \texttt{leave}\\
& \texttt{ret}
\end{aligned}
$$

Compiling Expressions

Now we translate IR expressions using the previously declared function $\mathcal{T}[\![t, \epsilon]\!]$.

$$
\begin{aligned}
\mathcal{T}[\![t, n]\!] &= \texttt{mov}\ \ t, n\\
\mathcal{T}[\![t, t']\!] &= \texttt{mov}\ \ t, t'\\
\mathcal{T}[\![t, \&L]\!] &= \texttt{lea}\ \ t, \texttt{QWORD PTR[}L\texttt{]}
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![t, [\epsilon]]\!] = \ & \mathcal{T}[\![t_1, \epsilon]\!]\\
& \texttt{mov}\ \ t, \texttt{QWORD PTR[}t_1\texttt{]}
\end{aligned}
$$

$$
\begin{aligned}
\mathcal{T}[\![t, e_1\ \mathsf{op}\ e_2]\!] = \ & \mathcal{T}[\![t_1, e_1]\!]\\
& \mathcal{T}[\![t_2, e_2]\!]\\
& \mathsf{compute}(t, t_1, \mathsf{op}, t_2)
\end{aligned}
$$

where $\mathsf{compute}(t, t_1, \mathsf{op}, t')$ is a helper function that outputs a sequence of abstract assembly statements that has the effect of computing the result of $t_1\,\mathsf{op}\,t_2$ and store the result in t. For basic

arithmetic and bitwise operations, this is easy:

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{op}, t_2) = {} & \mathsf{asm_of}(\mathsf{op})\ t_1, t_2 \\ & \texttt{mov}\ t, t_1\end{aligned} \qquad \text{where } \mathsf{op} \in \{\mathsf{add}, \mathsf{sub}, \mathsf{imul}, \mathsf{and}, \mathsf{or}, \mathsf{xor}\}$$

with the helper function $\mathsf{asm_of}(\mathsf{op})$ defined such that

$$\begin{aligned} \mathsf{asm_of}(\mathsf{add}) &= \texttt{add} & \mathsf{asm_of}(\mathsf{sub}) &= \texttt{sub} & \mathsf{asm_of}(\mathsf{mul}) &= \texttt{imul} \\ \mathsf{asm_of}(\mathsf{and}) &= \texttt{and} & \mathsf{asm_of}(\mathsf{or}) &= \texttt{or} & \mathsf{asm_of}(\mathsf{xor}) &= \texttt{xor} \end{aligned}$$

For operations involving comparisons, we need a longer sequence that involves the `setcc` instructions:

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{op}, t_2) = {} & \texttt{xor rax, rax} \\ & \texttt{cmp}\ t_1, t_2 \\ & \mathsf{asm_of}(\mathsf{op})\ \texttt{al} \\ & \texttt{mov}\ t, \texttt{rax}\end{aligned} \qquad \text{where } \mathsf{op} \in \{\mathsf{eq}, \mathsf{neq}, \mathsf{lt}, \mathsf{ult}, \mathsf{gt}, \mathsf{leq}, \mathsf{geq}\}$$

with

$$\begin{aligned} \mathsf{asm_of}(\mathsf{eq}) &= \texttt{sete} & \mathsf{asm_of}(\mathsf{neq}) &= \texttt{setne} & & \\ \mathsf{asm_of}(\mathsf{lt}) &= \texttt{setl} & \mathsf{asm_of}(\mathsf{gt}) &= \texttt{setg} & \mathsf{asm_of}(\mathsf{ult}) &= \texttt{setb} \\ \mathsf{asm_of}(\mathsf{geq}) &= \texttt{setge} & \mathsf{asm_of}(\mathsf{leq}) &= \texttt{setle} & & \end{aligned}$$

The remaining operations are the most involved, where we must follow a specific setup of registers before the computation.

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{div}, t_2) = {} & \texttt{mov rax}, t_1 \\ & \texttt{cqo} \\ & \texttt{idiv}\ t_2 \\ & \texttt{mov}\ t, \texttt{rax}\end{aligned}$$

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{mod}, t_2) = {} & \texttt{mov rax}, t_1 \\ & \texttt{cqo} \\ & \texttt{idiv}\ t_2 \\ & \texttt{mov}\ t, \texttt{rdx}\end{aligned}$$

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{hmul}, t_2) = {} & \texttt{mov rax}, t_1 \\ & \texttt{imul}\ t_2 \\ & \texttt{mov}\ t, \texttt{rdx}\end{aligned}$$

$$\begin{aligned}\mathsf{compute}(t, t_1, \mathsf{op}, t_2) = {} & \texttt{mov rcx}, t_2 \\ & \mathsf{asm_of}(\mathsf{op})\ t_1, \texttt{cl} \\ & \texttt{mov}\ t, t_1\end{aligned} \qquad \text{where } \mathsf{op} \in \{\mathsf{lshift}, \mathsf{rshift}, \mathsf{arshift}\}$$

where

$$\mathsf{asm_of}(\mathsf{rshift}) = \texttt{shr}, \qquad \mathsf{asm_of}(\mathsf{lshift}) = \texttt{shl}, \qquad \mathsf{asm_of}(\mathsf{arshift}) = \texttt{sar}.$$

Note that, by directly using the three shift operations provided by x86, we are implicitly requiring that the shift amount t' is within the range of a 8-bit two's complement number. However, since shifts are not natively supported by the source language, it is easy to enforce this within ourselves.

Trivial Register Allocation

We now describe the most trivial method to allocate registers and transform abstract assembly into pure assembly. First, we need to define the size of the stack frame of a function, denoted by $\mathsf{frame_size_of}(L)$ as mentioned. For a naive implementation, this is simple: take the number of distinct temporaries in the body of the function with label L, multiply it by 8 and round up to the nearest multiple of 16. Next, we enumerate the set of distinct temporaries from 1, so that we can write the set as $\{t_1, ..., t_n\}$. We then need to designate three concrete registers that will never be touched throughout the function body to which the values of temporaries will be moved from the stack. Unfortunately, there are only two caller-saved general purpose scratch registers `r10`, `r11` within x86-64. Therefore, one of them must be a function argument register, such as `r9`. However, by making `r9` the last register to be allocated, we can avoid corrupting function arguments before they are stored on the stack, since the only abstract assembly instruction in the function preamble that uses `r9` is a `mov` instruction with only one temporary operand. Thus we can define the following:

$$\mathsf{alloc_reg}(1) = \texttt{r11}, \qquad \mathsf{alloc_reg}(2) = \texttt{r10}, \qquad \mathsf{alloc_reg}(3) = \texttt{r9}.$$

Next, we can translate every abstract assembly instruction c that uses the temporaries $t_{i_1}, t_{i_2}, ..., t_{i_m}$ to the following sequence of instructions:

$$\begin{aligned}
\mathcal{T}[\![c]\!]_c = \ & \texttt{mov}\ \mathsf{alloc_reg}(1), \texttt{QWORD PTR[rbp-}8i_1\texttt{]} \\
& \vdots \\
& \texttt{mov}\ \mathsf{alloc_reg}(m), \texttt{QWORD PTR[rbp-}8i_m\texttt{]} \\
& c[\mathsf{alloc_reg}(1)/t_{i_1}, ..., \mathsf{alloc_reg}(m)/t_{i_m}] \\
& \texttt{mov QWORD PTR[rbp-}8i_1\texttt{]}, \mathsf{alloc_reg}(1) \\
& \vdots \\
& \texttt{mov QWORD PTR[rbp-}8i_m\texttt{]}, \mathsf{alloc_reg}(m)
\end{aligned}$$

where $c[\mathsf{alloc_reg}(1)/t_{i_1}, ..., \mathsf{alloc_reg}(m)/t_{i_m}]$ denotes the instruction where the temporary t_{i_1} is replaced by $\mathsf{alloc_reg}(1)$, and the temporary t_{i_2} is replaced by $\mathsf{alloc_reg}(2)$, etc.

More Optimized Tilings

Now we describe more optimized tilings of IR expressions and statements that make use of specialized x86 instructions. However, we must first point out an important issue regarding the formalization of translation functions: designing optimized tilings means that multiple tilings, and thus multiple translations, may be applicable to one IR node, making the translation functions non-deterministic. We won't explicitly formalize a non-deterministic translation scheme based on tilings, and delegate the job of selecting the right tiling to use for a certain IR node to a module in our compiler which we will not document formally here.

We begin with extra tiling for expressions. First, we use a more efficient method to set a register to 0:

$$\mathcal{T}[\![t, 0]\!] = \texttt{xor}\ \ t, t$$

Then, we notice that no work is to be done moving a register to the same register:

$$\mathcal{T}[\![t, t]\!] = \bullet$$

Next, we use more complicated addressing modes of the `mov` instructions to accelerate memory accesses:

$$\mathcal{T}[\![t, [\&L]]\!] = \texttt{mov}\ \ t, \texttt{QWORD PTR[}L\texttt{]}$$

$$\begin{aligned}\mathcal{T}[\![t, [\epsilon \text{ add } n]]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[t_1{+}n] \qquad -2^{31} \leq n \leq 2^{31}-1\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, [\epsilon \text{ mul } n]]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \qquad n = 1, 2, 4, 8 \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[n{*}t_1]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, [(\epsilon \text{ mul } n_1) \text{ add } n_2]]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \qquad n_1 = 1, 2, 4, 8; -2^{31} \leq n \leq 2^{31}-1 \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[n_1{*}t_1{+}n]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, [\epsilon_1 \text{ add } \epsilon_2]]\!] &= \mathcal{T}[\![t_1, \epsilon_1]\!] \\ &\quad \mathcal{T}[\![t_2, \epsilon_2]\!] \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[t_1{+}t_2]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, [\epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n)]]\!] &= \mathcal{T}[\![t_1, \epsilon_1]\!] \qquad n = 1, 2, 4, 8 \\ &\quad \mathcal{T}[\![t_2, \epsilon_2]\!] \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[t_1{+}n{*}t_2]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, [\epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n_1) \text{ add } n_2]]\!] &= \mathcal{T}[\![t_1, \epsilon_1]\!] \qquad n_1 = 1, 2, 4, 8 \text{ and } -2^{31} \leq n_2 \leq 2^{31}-1 \\ &\quad \mathcal{T}[\![t_2, \epsilon_2]\!] \\ &\quad \texttt{mov}\, t, \texttt{QWORD PTR}[t_1{+}n_1{*}t_2{+}n_2]\end{aligned}$$

Now we use shortcuts to translate binary expressions. First, we notice that negating a number can be done efficiently by the `neg` instruction:

$$\begin{aligned}\mathcal{T}[\![t, 0 \text{ sub } \epsilon]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \\ &\quad \texttt{neg}\, t_1 \\ &\quad \texttt{mov}\ t, t_1\end{aligned}$$

Then, we use the good-old trick where multiplications/divisions by a multiple of two is equivalent to shifting:

$$\begin{aligned}\mathcal{T}[\![t, \epsilon \text{ mul } 2^n]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \\ &\quad \texttt{shl}\ t_1, n \\ &\quad \texttt{mov}\ t, t_1 \\ \mathcal{T}[\![t, \epsilon \text{ div } 2^n]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \\ &\quad \texttt{sar}\ t_1, n \\ &\quad \texttt{mov}\ t, t_1\end{aligned}$$

Now we turn our attention to accelerating binary expressions through the `lea` instruction. This is similar to the optimizations about memory accesses that we have done above.

$$\begin{aligned}\mathcal{T}[\![t, \epsilon \text{ add } n]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \qquad -2^{31} \leq n \leq 2^{31}-1 \\ &\quad \texttt{lea}\, t, \texttt{QWORD PTR}[t_1{+}n]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, \epsilon \text{ mul } n]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \qquad n = 1, 2, 4, 8 \\ &\quad \texttt{lea}\, t, \texttt{QWORD PTR}[n{*}t_1]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, (\epsilon \text{ mul } n_1) \text{ add } n_2]\!] &= \mathcal{T}[\![t_1, \epsilon]\!] \qquad n_1 = 1, 2, 4, 8; -2^{31} \leq n \leq 2^{31}-1 \\ &\quad \texttt{lea}\, t, \texttt{QWORD PTR}[n_1{*}t_1{+}n]\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, \epsilon_1 \text{ add } \epsilon_2]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \texttt{lea}\, t, \texttt{QWORD PTR[}t_1\texttt{+}t_2\texttt{]}\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![t, \epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n)]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \texttt{lea}\, t, \texttt{QWORD PTR[}t_1\texttt{+}n\texttt{*}t_2\texttt{]}\end{aligned} \qquad n = 1, 2, 4, 8$$

$$\begin{aligned}\mathcal{T}[\![t, \epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n_1) \text{ add } n_2]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \texttt{lea}\, t, \texttt{QWORD PTR[}t_1\texttt{+}n_1\texttt{*}t_2\texttt{+}n_2\texttt{]}\end{aligned} \qquad n_1 = 1, 2, 4, 8,\ -2^{31} \leq n \leq 2^{31} - 1$$

Next, we turn our attention to optimizing entire statements. Using complex addressing modes in a similar manner to the above, we can speed up memory writes and have tiles like:

$$\begin{aligned}\mathcal{T}[\![[\&L] \leftarrow \epsilon]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \texttt{mov QWORD PTR[}L\texttt{]}, t_1\end{aligned}$$

$$\begin{aligned}\mathcal{T}[\![[\epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n)] \leftarrow \epsilon]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \mathcal{T}[\![t, \epsilon]\!] \\ & \texttt{mov QWORD PTR[}t_1\texttt{+}n_1\texttt{*}t_2\texttt{]}, t\end{aligned} \qquad n = 1, 2, 4, 8$$

$$\begin{aligned}\mathcal{T}[\![[\epsilon_1 \text{ add } (\epsilon_2 \text{ mul } n_1) \text{ add } n_2] \leftarrow \epsilon]\!] = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \mathcal{T}[\![t, \epsilon]\!] \\ & \texttt{mov QWORD PTR[}t_1\texttt{+}n_1\texttt{*}t_2\texttt{+}n_2\texttt{]}, t\end{aligned} \qquad n_1 = 1, 2, 4, 8, -2^{31} \leq n_2 \leq 2^{31} - 1$$

An astute reader may realize the above set of optimizations can be automatically achieved through constant propagation on the assembly level only using the `lea` and `mov` optimizations at the instruction-level. This is indeed the case. It is up to the implementer to decide whether to implement the optimization prematurely; regardless of the implementer's decision, it is important to remember that these statement-level optimizations does not come for free only using their expression-level counterparts.

The final important class of optimizations we consider are those involving conditional jumps. The insight is that, if we peek the guard of conditional jump statements, we can take advantage of the wide variety of condition codes of conditional jump to simplify computations.

$$\begin{aligned}\mathsf{cjump}\ (\epsilon_1\ \mathsf{op}\ \epsilon_2)\ L = \; & \mathcal{T}[\![t_1, \epsilon_1]\!] \\ & \mathcal{T}[\![t_2, \epsilon_2]\!] \\ & \texttt{cmp}\ t_1, t_2 \\ & \mathsf{jump_of(op)}\ .L\end{aligned} \qquad \mathsf{op} \in \{\mathsf{eq}, \mathsf{neq}, \mathsf{lt}, \mathsf{gt}, \mathsf{geq}, \mathsf{leq}, \mathsf{ult}\}$$

where the helper function jump_of(op) is defined as

$$\begin{aligned} \mathsf{jump_of(eq)} &= \texttt{jz} & \mathsf{jump_of(neq)} &= \texttt{jnz} & & \\ \mathsf{jump_of(lt)} &= \texttt{jl} & \mathsf{jump_of(gt)} &= \texttt{jg} & \mathsf{jump_of(ult)} &= \texttt{jb} \\ \mathsf{jump_of(geq)} &= \texttt{jge} & \mathsf{jump_of(leq)} &= \texttt{jle} & & \end{aligned}$$

In the case that one of the operands is zero, we can do even better using the test instruction, which computes a bitwise AND:

$$\begin{aligned}\mathsf{cjump}\ (\epsilon\ \mathsf{op}\ 0)\ L = \mathcal{T}[\![t, \epsilon]\!] \qquad & \mathsf{op} \in \{\mathsf{eq}, \mathsf{neq}, \mathsf{lt}, \mathsf{geq}\} \\ \texttt{test}\ t, t & \\ \mathsf{jump_of_test}(\mathsf{op})\ \texttt{.}L &\end{aligned}$$

where the helper function $\mathsf{jump_of_test}(\mathsf{op})$ is defined by

$$\mathsf{jump_of_test}(\mathsf{eq}) = \texttt{jz}, \quad \mathsf{jump_of_test}(\mathsf{neq}) = \texttt{jnz}, \quad \mathsf{jump_of_test}(\mathsf{lt}) = \texttt{js}, \quad \mathsf{jump_of_test}(\mathsf{geq}) = \texttt{jns}$$

Again, these instructions can benefit from complex addressing modes (e.g. x86-64 accepts instructions of form `cmp [rax], 2`), but again, as we have discussed earlier, it may be more preferable to allow a copy propagator to activate these transformations.

Equality Saturation

In order to make instruction selection more effective, we might want to employ a limited form of equality saturation. That is, we apply algebraic transformations to some equations so that they could be detected by instruction selection. The following needs to applied when the IR expression tree is created:

$$\begin{aligned}(\epsilon_1\ \mathsf{add}\ \epsilon_2)\ \mathsf{mul}\ n &\implies (\epsilon_1\ \mathsf{mul}\ n)\ \mathsf{plus}\ (\epsilon_2\ \mathsf{mul}\ n) && n = 1, 2, 4, 8 \\ \epsilon\ \mathsf{sub}\ n &\implies \epsilon\ \mathsf{add}\ (-n) && -(2^{31} - 1) \leq n \leq 2^{31} - 1\end{aligned}$$

We also need to detect, for example, all expressions semantically equivalent to $\epsilon_1\ \mathsf{add}\ (\epsilon_2\ \mathsf{mul}\ n_1)\ \mathsf{add}\ n_2$ modulo commutativity and associativity. Instead of applying commutativity and associativity laws as above, it is better to do so through a recursive traversal on IR expressions during code generation that detects the individual terms of the expressions above: $\epsilon_1, \epsilon_2\ \mathsf{mul}\ n_1, n_2$.